Homework 3
The Old Keypad

Julia Reynolds, Akiva Lippert

October 14, 2026

Prompt

Write a project report in PDF format. The report includes:

1. Please analyze the time complexity of your algorithm if users enter N digits. Specifically, the time cost for constructing the tree, the time cost for dictionary searching, and the cost for solving the entire problem of N digits of input.
2. Please search online references regarding the concept of Exhaustive Search. What is Exhaustive Search? Which part of this project falls into Exhaustive Search?
3. Please search online references regarding the concept of Branch and Bound. What is Branch and bound?
4. If you use the Prefix Tree to implement the dictionary, can you use the prefix tree property to accelerate your algorithm together with Branch and Bound? and How? please illustrate with a specific example and use a diagram if necessary.
5. Please include at least 5 different runs of your program with different input sequences. Please catch any errors so that your program will not crash.

1 Time Complexity

I use $|q|$ (read as "length of q") to refer to the number of digits in the query, *instead* of N.

1.1 Exhaustive Search

1.1.1 Dictionary Building

Exhaustive search uses a simple HashSet to store its dictionary, which is very lightweight and has an `add()` time complexity of amortized $O(1)$, leading to an amortized $O(n)$ time complexity to build the dictionary (where n is the number of words in the dictionary).

1.1.2 Dictionary Searching

C#'s HashSet `contains()` time complexity is amortized $O(1)$.

1.1.3 Search Algorithm

Since exhaustive search needs to explore the entire tree without terminating early, the time complexity of this search is in relation to the branching factor of the tree. There are 8 buttons, 2 of which correspond to 4 possibilities, the rest corresponding to 3, which gives a branching factor of $\frac{(2*4)+(3*6)}{8} = 3.25$. In other

words, each number read in leads to 3.25 branches in the tree to explore. And since every digit in q needs to be fully explored, this leads to a per-query time complexity of $O(3.25^{|q|})$.

1.2 Branch and Bound

1.2.1 Dictionary Building

Constructing the trie for Branch & Bound is somewhat costly because the algorithm needs to traverse the tree on each insert, and it needs to do that traversal for each word in the dictionary. This means that it builds and checks as it goes, with each traversal being $O(|w|)$ (where $|w|$ is the length of the word) and at n times, the building process is $O(n * |w|)$

1.2.2 Dictionary Searching

Navigating the trie to find a specific word would take at most $O(|w|)$ time, but it can terminate early if it reaches a leaf node before it reaches its search target. This is almost, but not quite, $O(1)$

1.2.3 Search Algorithm

Branch and bound has a search time complexity of $O(3.25^{|q|})$ for this problem since in the worst-case scenario nothing gets pruned early and multiple paths need to be explored. However, the average case complexity would be much better since you are likely to chop off a lot of the work. In the best case scenario it takes $|q|$ time by pruning every nonviable path as soon as possible.

1.3 Preprocessed Hashmap

This section is because Akiva is a stick in the mud who has strong opinions about Hashmaps

To me, this problem is poorly suited to the tree-based approach, and is better approached from the dictionary-end for a few reasons:

1. Plain Hashmaps (or generally: Dictionaries) have time complexities that can meet or beat the other algorithms given here
2. Hashmaps are very easy to implement and are often a native data structure to a programming language, which circumvents the need to build custom trees (which I find tedious at best)
3. It approaches the problem from the other end, which is also a fun learning exercise

1.3.1 Algorithmic Process Overview

Step	Process	Time Complexity	Description
1	Build the Hashmap	Amortized $O(n)$	Build the Hashmap *backwards* by generating keys as the numbers a word maps to, and then storing all the words that map to that number in a list as the value for the key
1a	Map a word	$O(1)$	For each letter of the word, map it to its keypad equivalent ($O(\|w\|)$, which is constant relative to the length of the word, and estimated to be at most 45)
1b	Insert into Hashmap	Amortized $O(1)$	If the key doesn't exist, add it to the hashmap (amortized $O(1)$), then add the word into the key's list of matches (also $O(1)$, assuming an ArrayList is used).
1c	Repeat for whole Dictionary	$O(n)$	Repeat 1a and 1b for each word in the dictionary of n words
2	Query the Hashmap	$O(1)$ per query	Uses the concatenated search number to grab the key of any matches, and returns the list attached to that key. No key found means no words match that input

1.3.2 Dictionary Building

Since there's no way to avoid touching every word in the provided dictionary once during building given the problem constraints, any structure build algorithm can't do better than $O(n)$ time (where n is the number of words in the dictionary). A Hashmap has amortized $O(1)$ insertions into the main map for each of the n entries in the dictionary, leading to $O(n)$ build times on the dictionary, though it's a much heavier/slower $O(n)$ than Exhaustive Search due to there being more constant-time calculations per n.

1.3.3 Dictionary Search *is* the Search Algorithm

Where this implementation does better is that the preprocessing done to the dictionary cuts search times to a Hashmap's `get()` function, which has a native $O(1)$ time complexity per-query. This algorithmic structure has the neat property where the search time complexity is not linked to $|q|$ at all.

1.3.4 Further Notes on Time Complexity

A quick Google search shows that estimates for the number of words in the English language don't reasonably exceed 1,000,000, and the longest word is likely about 45 characters, which can also be considered the longest query for this problem.

With those assumptions, the estimated lower bound for processing times on each of the dictionary sizes on a given CPU would be:

Dictionary Size CPU Speed	25,710 words	171,000 words	470,000 words	1,000,000 words
1.0 GHz	0.026ms	0.171ms	0.470ms	1.000ms
1.3 GHz	0.019ms	0.131ms	0.361ms	0.769ms
2.0 GHz	0.012ms	0.085ms	0.235ms	0.500ms
2.5 GHz	0.010ms	0.068ms	0.188ms	0.400ms
3.0 GHz	0.008ms	0.057ms	0.156ms	0.333ms
3.5 GHz	0.007ms	0.048ms	0.134ms	0.285ms
4.0 GHz	0.006ms	0.042ms	0.117ms	0.250ms
4.7 GHz	0.005ms	0.036ms	0.100ms	0.212ms

2 Exhaustive Search

Summarized from Wikipedia: Exhaustive Search is also known as Brute-Force Search, and it's the process of checking all options when searching for a given value [2]. If a problem allows for alternative search methods, Exhaustive Search is usually the slowest in terms of time complexity.

This project has two implementations of the search, and Exhaustive Search is one of them. It's implemented by recursively traversing each branch of a word tree until each node has been visited, checking along the way if a word is a match to the search conditions.

3 Branch and Bound

Summarized from Wikipedia: Branch-and-Bound (BnB) is an optimization method that helps to prune branching problems and limit their runtime. BnB checks the current search space (the "branch") against estimated upper and lower limits (the "bounds"), and if it falls outside of the bounds, it is discarded as a dead branch that cannot contain a viable solution. Effectively, BnB checks the search space to see if it *can* hold the solution before it checks if the search space *does* hold the solution [1].

BnB is utilized as the search method within the prefix-tree class. It prunes branches as it goes by pruning any branches that at somepoint don't have a corresponding child node with the desired character.

4 Prefix Tree

A prefix-tree, or trie, is used to store and retrieve strings. It is constructed by storing a set of characters that each link to a node that stores another set of characters and so on and so forth, so "every child node shares a common prefix with its parent node" [3]. You can then search for a string starting with the first character of the string and traveling down the path each character leads to. Each node stores a boolean that indicates whether the prefix is a word or not.

Prefix trees can be advantageous when used for tasks such as "autocomplete, spell checking, and IP routing" due to the way they store the characters [3].

Can you use the prefix tree property to accelerate your algorithm together with Branch and Bound?

Yes, prefix-trees work well with BnB due to the structure of the stored data. Because each Node of the tree can only lead to the next if there is an existing character with its associated Node stored there, it is easy to know which branches to prune. The branches without the desired child node have no way of possibly leading to a valid word, and therefore are pruned, reducing the amount of searching that needs to be done. Therefore a prefix-tree that implements the BnB algorithm to search it will run faster than finding all possible combinations with exhaustive search and only accessing the tree at the end of the process to verify the existence of each option.

For example using the tree to our advantage while searching using BnB, if we take the sequence 6572 ("okra"), we start by looking at the root. Since 6 corresponds to m, n, and o, we check for the existence of the corresponding children. Since there are valid words starting with each of those letters, we proceed with all paths. Then we reach 5, which corresponds to j, k, and l. This generates 9 combinations (mj, mk, ml, nj, nk, nl, oj, ok, ol), which depending on the words in your dictionary, 7 of these would likely be pruned since there aren't many, if any, valid words with those prefixes (though there are some acronyms I believe). We continue on with ok, and ol. 7 corresponds to p, q, r, and s, of which only okr is valid, so we continue with that, which leads us to okra.

Sadly, it doesn't appear that the word "okra" is in the dictionary we are using for this assignment, but the illustration still stands.

5 Program Runs

5.0.1 Important Notes

- These runs were reported from Akiva's personal computer running at a base CPU clock speed of 4.7GHz. Your results will vary depending on your CPU's clock speeds
- Windows cannot report system times below 100ns, so any result of 100ns should be interpreted as "100ns or less"
- All reported times seem to be a little slower than expected for the first 2-3 runs, which might be due to garbage collection, compiling time, other "warm-up" activities, or other unknown slowdowns that are unique to small-scale queries. I'm not certain of the true cause

5.1 Increasing Query Length

	Build Time	Query Time (ns)						
Algorithm	(ns)	465	7323	78683	288467	8745649	26657467	542727437
Exhaustive Search	**13,358,600**	1,600	8,500	19,700	36,900	157,900	466,300	2,319,800
Branch & Bound	27,534,200	4,900	8,200	1,500	1,100	1,100	1,600	1,400
Keypad Hashmap	26,454,300	**200**	**400**	**500**	**100**	**100**	**100**	**100**

The dictionary build times are significant for all 3, but Exhaustive Search tends to build its tree in half the time of Branch & Bound or Keypad Hashmap because the underlying HashSet is much faster to build and there's no preprocessing steps.

Branch & Bound and Keypad Hashmap both have larger overheads, but the Keypad Hashmap comes out slightly ahead. This is due to them both using heavier data structures than Exhaustive Search, with Branch & Bound's custom-built node-based prefix tree being heavier than a `Dictionary<string, List<string>`.

These queries, except the first one, all increase in query length but not results length. This (mostly) isolates the variables so we can compare the observed time complexity to the theoretical ones. Exhaustive search, as expected, scales with the query length with a time complexity of $O(n^{|q|})$. Branch & Bound seems to have a stable runtime of 1-1.5k ns on my system, and it should be $O(|q|)$, which is almost constant. Keypad Hashmap has the best query runtime, bottoming out at a measured 100ns, meaning it could be running faster, due to the $O(1)$ lookups of a Hashmap.

5.1.1 Input: 465

Algorithm	Words Found	Time (ns)
Exhaustive Search	gol, ink, inl	1,600
Branch & Bound	gol, ink, inl	4,900
Keypad Hashmap	gol, ink, inl	**200**

5.1.2 Input: 7323

Algorithm	Words Found	Time (ns)
Exhaustive Search	read	8,500
Branch & Bound	read	8,200
Keypad Hashmap	read	**400**

5.1.3 Input: 78683

Algorithm	Words Found	Time (ns)
Exhaustive Search	quote	19,700
Branch & Bound	quote	1,500
Keypad Hashmap	quote	**500**

5.1.4 Input: 288467

Algorithm	Words Found	Time (ns)
Exhaustive Search	author	36,900
Branch & Bound	author	1,100
Keypad Hashmap	author	**100**

5.1.5 Input: 8745649

Algorithm	Words Found	Time (ns)
Exhaustive Search	trilogy	157,900
Branch & Bound	trilogy	1,100
Keypad Hashmap	trilogy	**100**

5.1.6 Input: 26657467

Algorithm	Words Found	Time (ns)
Exhaustive Search	bookshop	466,300
Branch & Bound	bookshop	1,600
Keypad Hashmap	bookshop	**100**

5.1.7 Input: 542727437

Algorithm	Words Found	Time (ns)
Exhaustive Search	libraries	2,319,800
Branch & Bound	libraries	1,400
Keypad Hashmap	libraries	**100**

5.2 Increasing Query Results

Algorithm	Build Time (ns)	Query Time (ns) 2628	2633	2662	7243	2273
Exhaustive Search	**12,859,800**	4,100	8,200	3,600	6,200	5,200
Branch & Bound	27,223,400	7,200	8,200	1,400	1,400	1,400
Keypad Hashmap	25,021,600	**200**	**400**	**100**	**100**	**100**

This doesn't add too much extra information, but it does show that the algorithms seem to be stable on a trivial-sized dictionary for any amount of results for a given query, which I think is nice.

5.2.1 Input: 2628

Algorithm	Words Found	Time (ns)
Exhaustive Search	boat, coat	4,100
Branch & Bound	boat, coat	7,200
Keypad Hashmap	boat, coat	**200**

5.2.2 Input: 2633

Algorithm	Words Found	Time (ns)
Exhaustive Search	bode, code, coed	8,200
Branch & Bound	bode, code, coed	8,200
Keypad Hashmap	bode, code, coed	**400**

5.2.3 Input: 2662

Algorithm	Words Found	Time (ns)
Exhaustive Search	anna, bomb, coma, comb	3,600
Branch & Bound	anna, bomb, coma, comb	1,400
Keypad Hashmap	anna, bomb, coma, comb	**100**

5.2.4 Input: 7243

Algorithm	Words Found	Time (ns)
Exhaustive Search	page, paid, rage, raid, sage, said	6,200
Branch & Bound	page, paid, rage, raid, sage, said	1,400
Keypad Hashmap	page, paid, rage, raid, sage, said	**100**

5.2.5 Input: 2273

Algorithm	Words Found	Time (ns)
Exhaustive Search	acre, bare, base, cape, card, care, case	5,200
Branch & Bound	acre, bare, base, cape, card, care, case	1,400
Keypad Hashmap	acre, bare, base, cape, card, care, case	**100**

References

[1] Wikipedia contributors. Branch and bound. *Wikipedia*, 2026.

[2] Wikipedia contributors. Brute-force search. *Wikipedia*, 2026.

[3] Wikipedia contributors. Trie. *Wikipedia*, 2026.